

# 공통 컴포넌트 사용 예시

이 문서는 ITCEN Solution 프로젝트에서 만든 공통 컴포넌트들의 실제 사용 예시와 문제 해결 사례를 제공합니다.

## 최신 업데이트 사항

### DataGrid 높이 문제 해결 (2024.12)

- MUI X DataGrid v7.7.1에서 발생하는 높이 관련 오류 해결
- `autoHeight` 속성 사용으로 모든 높이 문제 해결
- 탭 시스템 내부에서도 안정적으로 동작

### 글로벌 Toast 시스템 (2024.12)

- 전역 Toast 관리 시스템 구축
- 성공, 에러, 경고, 정보 메시지 일관된 표시
- 사용자 친화적인 피드백 시스템

### 글로벌 Loading 상태 관리 (2024.12)

- 전역 Loading 상태 관리 시스템 구축
- 비동기 작업 중 로딩 표시 자동화
- 사용자 경험 개선

### 서버 통합 컴포넌트 (2024.12)

- ServerDataGrid: 서버 사이드 데이터 그리드
- ServerFileUpload: 실제 서버 파일 업로드
- 실무에서 바로 사용 가능한 컴포넌트들

## 1. Button 컴포넌트

### 기본 사용법

```
import { Button } from '@shared/components/ui';

// 기본 버튼
<Button onClick={() => console.log('클릭됨')}>
```

```

    기본 버튼
</Button>

// 다양한 스타일
<Button variant="contained" color="primary">Primary</Button>
<Button variant="outlined" color="secondary">Secondary</Button>
<Button variant="text" color="error">Error</Button>

// 크기 조절
<Button size="small">작은 버튼</Button>
<Button size="medium">중간 버튼</Button>
<Button size="large">큰 버튼</Button>

// 로딩 상태
<Button loading>로딩 중...</Button>

// 아이콘과 함께
<Button startIcon={<SaveIcon />}>저장</Button>
<Button endIcon={<SendIcon />}>전송</Button>

```

## 실제 사용 예시

```

const MyPage: React.FC = () => {
  const [loading, setLoading] = useState(false);

  const handleSave = async () => {
    setLoading(true);
    try {
      await saveData();
      alert('저장되었습니다. ');
    } catch (error) {
      alert('저장 실패 ');
    } finally {
      setLoading(false);
    }
  };

  return (
    <Box sx={{ display: 'flex', gap: 2 }}>
      <Button
        variant='contained'
        color='primary'
        loading={loading}
        onClick={handleSave}
        startIcon={<SaveIcon />}
      >
        저장
      </Button>
    </Box>
  );
};

```

```

        <Button variant='outlined' onClick={() => navigate('/list')}>
            목록으로
        </Button>
    </Box>
);
};

```

## 2. Select 컴포넌트

### 기본 사용법

```

import { Select, SelectOption } from '@shared/components/ui';

const options: SelectOption[] = [
    { value: '1', label: '옵션 1' },
    { value: '2', label: '옵션 2' },
    { value: '3', label: '옵션 3' },
];

// 기본 셀렉트
<Select
    label="카테고리"
    options={options}
    value={selectedValue}
    onChange={(value) => setSelectedValue(value)}
    placeholder="선택해주세요"
/>

// 다중 선택
<Select
    label="다중 선택"
    options={options}
    multiple
    value={selectedValues}
    onChange={(values) => setSelectedValues(values)}
/>

// 그룹핑
const groupedOptions: SelectOption[] = [
    { value: '1', label: '서울', group: '수도권' },
    { value: '2', label: '인천', group: '수도권' },
    { value: '3', label: '부산', group: '영남권' },
    { value: '4', label: '대구', group: '영남권' },
];

<Select

```

```

    label="지역"
    options={groupedOptions}
    groupBy="group"
    value={selectedRegion}
    onChange={(value) => setSelectedRegion(value)}
  />

```

## 공통코드와 연동

```

const CategorySelect: React.FC = () => {
  const [categories, setCategories] = useState<SelectOption[]>([]);
  const [selectedCategory, setSelectedCategory] = useState<string>('');

  useEffect(() => {
    // 공통코드에서 카테고리 목록 가져오기
    const codes = getCodesArray();
    const categoryOptions = getCategoryGroupCodes().map(code => ({
      value: code.code,
      label: code.codeName,
    }));
    setCategories(categoryOptions);
  }, []);

  return (
    <Select
      label='카테고리'
      options={categories}
      value={selectedCategory}
      onChange={value => setSelectedCategory(value as string)}
      required
      fullWidth
    />
  );
};

```

## 3. ComboBox 컴포넌트

### 기본 사용법

```

import { ComboBox, SelectOption } from '@shared/components/ui';

const userOptions: SelectOption[] = [
  { value: '1', label: '김철수' },
  { value: '2', label: '이영희' },
  { value: '3', label: '박민수' },
];

```

```
// 기본 콤보박스
<ComboBox
  label="사용자 선택"
  options={userOptions}
  value={selectedUser}
  onChange={(value) => setSelectedUser(value)}
  placeholder="사용자를 검색하세요"
/>

// 자유 입력 가능
<ComboBox
  label="태그"
  options={tagOptions}
  value={selectedTags}
  onChange={(values) => setSelectedTags(values)}
  multiple
  freeSolo
  placeholder="태그를 입력하거나 선택하세요"
/>
```

## 비동기 데이터 로딩

```
const AsyncComboBox: React.FC = () => {
  const [options, setOptions] = useState<SelectOption[]>([]);
  const [loading, setLoading] = useState(false);
  const [inputValue, setInputValue] = useState('');

  const loadOptions = async (searchTerm: string) => {
    if (searchTerm.length < 2) return;

    setLoading(true);
    try {
      const response = await api.searchUsers(searchTerm);
      const userOptions = response.data.map(user => ({
        value: user.id,
        label: `${user.name} (${user.email})`,
      }));
      setOptions(userOptions);
    } catch (error) {
      console.error('사용자 검색 실패:', error);
    } finally {
      setLoading(false);
    }
  };

  const handleInputChange = (value: string) => {
    setInputValue(value);
  };
}
```

```

    loadOptions(value);
  };

  return (
    <ComboBox
      label='사용자 검색'
      options={options}
      loading={loading}
      onInputChange={handleInputChange}
      placeholder='이름 또는 이메일로 검색'
      noOptionsText='검색 결과가 없습니다'
      loadingText='검색 중...'
    />
  );
};

```

## 4. DataGrid 컴포넌트

### 기본 사용법

```

import { DataGrid, DataGridColumn } from '@shared/components/ui';

interface User {
  id: number;
  name: string;
  email: string;
  department: string;
  createdAt: string;
}

const userColumns: DataGridColumn<User>[] = [
  {
    field: 'id',
    headerName: 'ID',
    width: 70,
    sortable: true,
  },
  {
    field: 'name',
    headerName: '이름',
    width: 120,
    sortable: true,
  },
  {
    field: 'email',
    headerName: '이메일',
    width: 200,

```

```

    flex: 1,
  },
  {
    field: 'department',
    headerName: '부서',
    width: 150,
  },
  {
    field: 'createdAt',
    headerName: '등록일',
    width: 120,
    renderCell: ({ value }) => {
      return new Date(value).toLocaleDateString();
    },
  },
],
];

```

// 기본 데이터그리드

```

<DataGrid
  data={users}
  columns={userColumns}
  height={400}
  onClick={row => console.log('선택된 사용자:', row)}
/>

```

// autoHeight 속성 사용 (권장)

```

<DataGrid
  data={users}
  columns={userColumns}
  autoHeight // 자동 높이 조정 - MUI X v7.7.1 권장
  onClick={row => console.log('선택된 사용자:', row)}
/>

```

## 선택 가능한 그리드

```

const SelectableDataGrid: React.FC = () => {
  const [selectedRows, setSelectedRows] = useState<number[]>([]);
  const [selectedData, setSelectedData] = useState<User[]>([]);

  const handleSelectionChange = (rowIds: number[], data: User[]) => {
    setSelectedRows(rowIds);
    setSelectedData(data);
  };

  const handleBulkDelete = async () => {
    if (selectedRows.length === 0) {
      alert('삭제할 항목을 선택하세요.');
```

```

    }

    if (confirm(`${selectedRows.length}개 항목을 삭제하시겠습니까?`)) {
      try {
        await api.deleteUsers(selectedRows);
        // 목록 새로고침
        fetchUsers();
        setSelectedRows([]);
      } catch (error) {
        alert('삭제 실패');
      }
    }
  };

  return (
    <Box>
      <Box sx={{ mb: 2 }}>
        <Button
          variant='contained'
          color='error'
          disabled={selectedRows.length === 0}
          onClick={handleBulkDelete}
        >
          선택 항목 삭제 ({selectedRows.length})
        </Button>
      </Box>

      <DataGrid
        data={users}
        columns={userColumns}
        selectable
        multiSelect
        selectedRows={selectedRows}
        onRowSelectionChange={handleSelectionChange}
        height={500}
      />
    </Box>
  );
};

```

## 페이지네이션이 있는 그리드

```

const PaginatedDataGrid: React.FC = () => {
  const [users, setUsers] = useState<User[]>([]);
  const [page, setPage] = useState(1);
  const [pageSize, setPageSize] = useState(10);
  const [totalItems, setTotalItems] = useState(0);
  const [loading, setLoading] = useState(false);

```



```

const fetchUsers = async (pageNum: number, size: number) => {
  setLoading(true);
  try {
    const response = await api.getUsers({
      page: pageNum,
      size: size,
    });
    setUsers(response.data.content);
    setTotalItems(response.data.totalElements);
  } catch (error) {
    console.error('사용자 목록 조회 실패:', error);
  } finally {
    setLoading(false);
  }
};

useEffect(() => {
  fetchUsers(page, pageSize);
}, [page, pageSize]);

const handlePageChange = (newPage: number) => {
  setPage(newPage);
};

const handlePageSizeChange = (newPageSize: number) => {
  setPageSize(newPageSize);
  setPage(1); // 페이지 크기 변경 시 첫 페이지로
};

return (
  <DataGrid
    data={users}
    columns={userColumns}
    loading={loading}
    pagination={{
      page,
      pageSize,
      totalItems,
      totalPages: Math.ceil(totalItems / pageSize),
      onPageChange: handlePageChange,
      onPageSizeChange: handlePageSizeChange,
      pageSizeOptions: [5, 10, 25, 50],
    }}
    height={600}
    onClick={row => navigate(`/users/${row.id}`)}
  />
);
};

```

## 액션 버튼이 있는 그리드

```
const ActionDataGrid: React.FC = () => {
  const userColumnsWithActions: DataGridColumn<User>[] = [
    ...userColumns,
    {
      field: 'actions',
      headerName: '작업',
      width: 150,
      sortable: false,
      filterable: false,
      renderCell: ({ row }) => (
        <Box sx={{ display: 'flex', gap: 1 }}>
          <Button size='small' variant='outlined' onClick={() => handleEdit(row)}>
            수정
          </Button>
          <Button size='small' variant='outlined' color='error' onClick={() => handleDelete(row)}>
            삭제
          </Button>
        </Box>
      ),
    },
  ];

  const handleEdit = (user: User) => {
    navigate(`/users/${user.id}/edit`);
  };

  const handleDelete = async (user: User) => {
    if (confirm(`${user.name}님을 삭제하시겠습니까?`)) {
      try {
        await api.deleteUser(user.id);
        fetchUsers(); // 목록 새로고침
      } catch (error) {
        alert('삭제 실패');
      }
    }
  };

  return (
    <DataGrid data={users} columns={userColumnsWithActions} autoHeight disableRowSelection0
    >
  );
};
```

## 서버 연동 DataGrid (ServerDataGrid)

```

import { ServerDataGrid } from '@shared/components/ui/data-display';
import { useToast, useLoading } from '@shared/components/ui/feedback';

const ServerDataGridExample: React.FC = () => {
  const { showToast } = useToast();
  const { showLoading, hideLoading } = useLoading();

  const userColumns: DataGridColumn<User>[] = [
    {
      field: 'id',
      headerName: 'ID',
      width: 70,
      sortable: true,
    },
    {
      field: 'name',
      headerName: '이름',
      width: 120,
      sortable: true,
    },
    {
      field: 'email',
      headerName: '이메일',
      width: 200,
      flex: 1,
    },
    {
      field: 'department',
      headerName: '부서',
      width: 150,
    },
    {
      field: 'status',
      headerName: '상태',
      width: 100,
      renderCell: ({ value }) => (
        <Chip
          label={value === 'active' ? '활성' : '비활성'}
          color={value === 'active' ? 'success' : 'default'}
          size='small'
        />
      ),
    },
  ];

  const handleError = (error: Error) => {
    console.error('DataGrid 오류:', error);
    showToast('데이터 로딩 중 오류가 발생했습니다.', 'error');
  };

```

```

const handleLoadingChange = (loading: boolean) => {
  if (loading) {
    showLoading('데이터 로딩 중...');
  } else {
    hideLoading();
  }
};

return (
  <ServerDataGrid
    apiUrl='/api/users'
    columns={userColumns}
    queryParams={{ status: 'active' }}
    transformData={data => data.content || data} // 서버 응답 변환
    onError={handleError}
    onLoadingChange={handleLoadingChange}
    autoHeight
    pagination={{
      pageSize: 10,
      serverSide: true,
    }}
    searchable
    filterable
    sortable
    toolbar
    onClick={row => {
      console.log('선택된 사용자:', row);
      navigate(`/users/${row.id}`);
    }}
  />
);
};

```

## 탭 시스템 내부에서 DataGrid 사용

```

import { DataGrid } from '@shared/components/ui/data-display';
import { Tabs, TabContainer, TabContent } from '@shared/components/ui/navigation';

const TabsWithDataGrid: React.FC = () => {
  const tabItems = [
    {
      id: 'users',
      label: '사용자 목록',
      content: (
        <Box sx={{ p: 2 }}>
          <DataGrid
            data={users}

```

```

        columns={userColumns}
        autoHeight // 탭 내부에서도 안정적으로 동작
        searchable
        filterable
      />
    </Box>
  ),
},
{
  id: 'admins',
  label: '관리자 목록',
  content: (
    <Box sx={{ p: 2 }}>
      <DataGrid data={admins} columns={adminColumns} autoHeight searchable filterable />
    </Box>
  ),
},
],
];

return (
  <Box sx={{ display: 'flex', flexDirection: 'column', height: '100%' }}>
    <Tabs items={tabItems} defaultValue='users' />
  </Box>
);
};

```

## DataGrid 높이 문제 해결 사례

```

// ❌ 문제가 있는 코드 (MUI X v7.7.1)
const ProblematicDataGrid: React.FC = () => {
  return (
    <Box sx={{ display: 'flex', flexDirection: 'column' }}>
      <DataGrid
        data={data}
        columns={columns}
        height='100%' // 부모 높이가 불명확할 때 문제 발생
      />
    </Box>
  );
};

// ✅ 해결된 코드 (autoHeight 사용)
const FixedDataGrid: React.FC = () => {
  return (
    <Box sx={{ display: 'flex', flexDirection: 'column' }}>
      <DataGrid
        data={data}
        columns={columns}

```

```

        autoHeight // 자동 높이 조정으로 문제 해결
      />
    </Box>
  );
};

// ✅ 고정 높이 사용하는 경우
const FixedHeightDataGrid: React.FC = () => {
  return (
    <Box sx={{ height: 600, display: 'flex', flexDirection: 'column' }}>
      <DataGrid
        data={data}
        columns={columns}
        height={600} // 명확한 고정 높이
      />
    </Box>
  );
};

```

## 5. 글로벌 Toast 시스템

### 기본 Toast 사용법

```

import { useToast } from '@shared/components/ui/feedback';

const ToastExample: React.FC = () => {
  const { showToast } = useToast();

  const handleSuccess = () => {
    showToast('작업이 성공적으로 완료되었습니다!', 'success');
  };

  const handleError = () => {
    showToast('오류가 발생했습니다. 다시 시도해주세요.', 'error');
  };

  const handleWarning = () => {
    showToast('주의: 이 작업은 되돌릴 수 없습니다.', 'warning');
  };

  const handleInfo = () => {
    showToast('새로운 업데이트가 있습니다.', 'info');
  };

  return (
    <Box sx={{ display: 'flex', gap: 2 }}>
      <Button onClick={handleSuccess} variant='contained' color='success'>

```

```

        성공 메시지
      </Button>
      <Button onClick={handleError} variant='contained' color='error'>
        에러 메시지
      </Button>
      <Button onClick={handleWarning} variant='contained' color='warning'>
        경고 메시지
      </Button>
      <Button onClick={handleInfo} variant='contained' color='info'>
        정보 메시지
      </Button>
    </Box>
  );
};

```

## 실제 API 호출에서 Toast 사용

```

const ApiWithToast: React.FC = () => {
  const { showToast } = useToast();
  const { showLoading, hideLoading } = useLoading();

  const handleSaveUser = async (userData: User) => {
    showLoading('사용자 정보를 저장하는 중...');
    try {
      await api.saveUser(userData);
      showToast('사용자 정보가 성공적으로 저장되었습니다.', 'success');
    } catch (error) {
      showToast('사용자 정보 저장에 실패했습니다.', 'error');
    } finally {
      hideLoading();
    }
  };

  const handleDeleteUser = async (userId: string) => {
    if (!confirm('정말로 삭제하시겠습니까?')) return;

    showLoading('사용자를 삭제하는 중...');
    try {
      await api.deleteUser(userId);
      showToast('사용자가 성공적으로 삭제되었습니다.', 'success');
    } catch (error) {
      showToast('사용자 삭제에 실패했습니다.', 'error');
    } finally {
      hideLoading();
    }
  };

  return (

```

```

    <Box>
      <Button onClick={() => handleSaveUser(mockUser)}>사용자 저장</Button>
      <Button onClick={() => handleDeleteUser('123')}>사용자 삭제</Button>
    </Box>
  );
};

```

## 6. 글로벌 Loading 시스템

### 기본 Loading 사용법

```

import { useLoading } from '@shared/components/ui/feedback';

const LoadingExample: React.FC = () => {
  const { showLoading, hideLoading, isLoading } = useLoading();

  const handleLongTask = async () => {
    showLoading('데이터를 처리하는 중...');
    try {
      // 긴 작업 시뮬레이션
      await new Promise(resolve => setTimeout(resolve, 3000));
      console.log('작업 완료');
    } finally {
      hideLoading();
    }
  };

  const handleQuickTask = async () => {
    showLoading(); // 메시지 없이 로딩만 표시
    try {
      await new Promise(resolve => setTimeout(resolve, 1000));
      console.log('빠른 작업 완료');
    } finally {
      hideLoading();
    }
  };

  return (
    <Box sx={{ display: 'flex', gap: 2 }}>
      <Button onClick={handleLongTask} disabled={isLoading}>
        긴 작업 실행
      </Button>
      <Button onClick={handleQuickTask} disabled={isLoading}>
        빠른 작업 실행
      </Button>
    </Box>
  );
};

```



```
);  
};
```

## 파일 업로드와 Loading 연동

```
const FileUploadWithLoading: React.FC = () => {  
  const { showLoading, hideLoading } = useLoading();  
  const { showToast } = useToast();  
  
  const handleFileUpload = async (files: File[]) => {  
    showLoading(`${files.length}개 파일을 업로드하는 중...`);  
    try {  
      const formData = new FormData();  
      files.forEach(file => formData.append('files', file));  
  
      const response = await api.uploadFiles(formData);  
      showToast(`${files.length}개 파일이 성공적으로 업로드되었습니다.`, 'success');  
    } catch (error) {  
      showToast('파일 업로드에 실패했습니다.', 'error');  
    } finally {  
      hideLoading();  
    }  
  };  
  
  return (  
    <ServerFileUpload  
      uploadUrl='/api/files/upload'  
      onUpload={handleFileUpload}  
      multiple  
      maxSize={10 * 1024 * 1024} // 10MB  
      acceptedTypes={['image/*', 'application/pdf']}  
    />  
  );  
};
```

## 7. 서버 파일 업로드 (ServerFileUpload)

### 기본 서버 파일 업로드

```
import { ServerFileUpload } from '@shared/components/ui/form';  
  
const ServerFileUploadExample: React.FC = () => {  
  const handleSuccess = (uploadedFiles: UploadedFile[]) => {  
    console.log('업로드된 파일들:', uploadedFiles);  
  };  
};
```

```

const handleError = (error: Error) => {
  console.error('업로드 오류:', error);
};

return (
  <ServerFileUpload
    uploadUrl='/api/files/upload'
    onSuccess={handleSuccess}
    onError={handleError}
    multiple
    maxSize={5 * 1024 * 1024} // 5MB
    acceptedTypes={['image/*', 'application/pdf', '.docx', '.xlsx']}
    dropzoneText='파일을 드래그하거나 클릭하여 업로드하세요'
  />
);
};

```

## 이미지 전용 업로드

```

const ImageUploadExample: React.FC = () => {
  const [uploadedImages, setUploadedImages] = useState<UploadedFile[]>([]);

  const handleImageUpload = (files: UploadedFile[]) => {
    setUploadedImages(prev => [...prev, ...files]);
  };

  return (
    <Box>
      <ServerFileUpload
        uploadUrl='/api/images/upload'
        onSuccess={handleImageUpload}
        acceptedTypes={['image/*']}
        multiple
        maxFiles={5}
        maxSize={2 * 1024 * 1024} // 2MB
        preview
        dropzoneText='이미지를 업로드하세요'
      />

      {uploadedImages.length > 0 && (
        <Box sx={{ mt: 2 }}>
          <Typography variant='h6'>업로드된 이미지:</Typography>
          <Box sx={{ display: 'flex', gap: 1, flexWrap: 'wrap', mt: 1 }}>
            {uploadedImages.map((file, index) => (
              <img
                key={index}
                src={file.url}
                alt={file.name}

```

```

        style={{ width: 100, height: 100, objectFit: 'cover', borderRadius: 4 }}
      />
    )}}
  </Box>
</Box>
)}
</Box>
);
};

```

## 8. 조합해서 사용하기

### 검색 폼과 결과 그리드

```

const UserManagementPage: React.FC = () => {
  const [users, setUsers] = useState<User[]>([]);
  const [searchForm, setSearchForm] = useState({
    name: '',
    department: '',
    status: '',
  });
  const [loading, setLoading] = useState(false);

  const departmentOptions: SelectOption[] = [
    { value: 'dev', label: '개발팀' },
    { value: 'design', label: '디자인팀' },
    { value: 'pm', label: '기획팀' },
  ];

  const statusOptions: SelectOption[] = [
    { value: 'active', label: '활성' },
    { value: 'inactive', label: '비활성' },
  ];

  const handleSearch = async () => {
    setLoading(true);
    try {
      const response = await api.searchUsers(searchForm);
      setUsers(response.data);
    } catch (error) {
      console.error('검색 실패:', error);
    } finally {
      setLoading(false);
    }
  };

  return (

```

```

<Box sx={{ p: 3 }}>
  { /* 검색 폼 */ }
  <Paper sx={{ p: 2, mb: 3 }}>
    <Typography variant='h6' gutterBottom>
      사용자 검색
    </Typography>

    <Box sx={{ display: 'flex', gap: 2, mb: 2 }}>
      <TextField
        label='이름'
        value={searchForm.name}
        onChange={e =>
          setSearchForm(prev => ({
            ...prev,
            name: e.target.value,
          }))
        }
        sx={{ flex: 1 }}
      />

      <Select
        label='부서'
        options={departmentOptions}
        value={searchForm.department}
        onChange={value =>
          setSearchForm(prev => ({
            ...prev,
            department: value as string,
          }))
        }
        sx={{ flex: 1 }}
        placeholder='전체'
      />

      <Select
        label='상태'
        options={statusOptions}
        value={searchForm.status}
        onChange={value =>
          setSearchForm(prev => ({
            ...prev,
            status: value as string,
          }))
        }
        sx={{ flex: 1 }}
        placeholder='전체'
      />
    </Box>

    <Box sx={{ display: 'flex', gap: 1 }}>

```

```

        <Button variant='contained' onClick={handleSearch} loading={loading}>
            검색
        </Button>

        <Button
            variant='outlined'
            onClick={() => setSearchForm({ name: '', department: '', status: '' })}
        >
            초기화
        </Button>
    </Box>
</Paper>

{ /* 결과 그리드 */ }
<DataGrid
    data={users}
    columns={userColumns}
    loading={loading}
    height={500}
    onClick={row => navigate(`/users/${row.id}`)}
    toolbar
    searchable
    exportable
/>
</Box>
);
};

```

이렇게 공통 컴포넌트들을 조합하여 일관성 있고 사용하기 쉬운 UI를 구성할 수 있습니다.

## 6. Feedback 컴포넌트들

### Modal 컴포넌트

```

const ModalExample: React.FC = () => {
    const [isOpen, setIsOpen] = useState(false);
    const [isFormOpen, setIsFormOpen] = useState(false);
    const [isFullScreenOpen, setIsFullScreenOpen] = useState(false);

    // 기본 모달
    const BasicModal = () => (
        <Modal
            open={isOpen}
            onClose={() => setIsOpen(false)}
            title='기본 모달'
            maxWidth='sm'
            actions={

```

```

    <Box sx={{ display: 'flex', gap: 1 }}>
      <Button onClick={() => setIsOpen(false)}>취소</Button>
      <Button variant='contained' onClick={() => setIsOpen(false)}>
        확인
      </Button>
    </Box>
  }
>
<Typography>
  이것은 기본 모달의 예시입니다. 타이틀, 콘텐츠, 액션 버튼을 포함할 수 있습니다.
</Typography>
</Modal>
);

```

// 폼 모달

```

const FormModal = () => (
  <Modal
    open={isFormOpen}
    onClose={() => setIsFormOpen(false)}
    title='사용자 추가'
    maxWidth='md'
    actions={
      <Box sx={{ display: 'flex', gap: 1 }}>
        <Button onClick={() => setIsFormOpen(false)}>취소</Button>
        <Button variant='contained' onClick={handleSubmit}>
          저장
        </Button>
      </Box>
    }
  >
    <Box sx={{ display: 'flex', flexDirection: 'column', gap: 2, pt: 2 }}>
      <TextField label='이름' required fullWidth />
      <TextField label='이메일' type='email' required fullWidth />
      <Select
        label='부서'
        options={departmentOptions}
        fullWidth
        placeholder='부서를 선택하세요'
      />
      <TextField label='전화번호' fullWidth />
    </Box>
  </Modal>
);

```

// 전체 화면 모달

```

const FullScreenModal = () => (
  <Modal
    open={isFullScreenOpen}
    onClose={() => setIsFullScreenOpen(false)}
    title='전체 화면 모달'
  >

```

```

    fullScreen
    actions={
      <Box sx={{ display: 'flex', gap: 1 }}>
        <Button onClick={() => setIsFullScreenOpen(false)}>닫기</Button>
      </Box>
    }
  >
  <Typography variant='h6' gutterBottom>
    전체 화면 모달 콘텐츠
  </Typography>
  <Typography>
    이 모달은 전체 화면으로 표시됩니다. 모바일에서 더 나은 사용자 경험을 제공합니다.
  </Typography>
</Modal>
);

const handleSubmit = () => {
  // 폼 제출 로직
  console.log('Form submitted');
  setIsFormOpen(false);
};

return (
  <Box sx={{ display: 'flex', gap: 2 }}>
    <Button variant='outlined' onClick={() => setIsOpen(true)}>
      기본 모달
    </Button>
    <Button variant='outlined' onClick={() => setIsFormOpen(true)}>
      폼 모달
    </Button>
    <Button variant='outlined' onClick={() => setIsFullScreenOpen(true)}>
      전체 화면 모달
    </Button>

    <BasicModal />
    <FormModal />
    <FullScreenModal />
  </Box>
);
};

```

## Alert 컴포넌트

```

const AlertExample: React.FC = () => {
  const [alerts, setAlerts] = useState({
    success: true,
    error: true,
    warning: true,
  });

```

```

    info: true,
  });

const handleClose = (type: string) => {
  setAlerts(prev => ({ ...prev, [type]: false }));
};

const handleResetAll = () => {
  setAlerts({
    success: true,
    error: true,
    warning: true,
    info: true,
  });
};

return (
  <Box sx={{ display: 'flex', flexDirection: 'column', gap: 2 }}>
    <Button variant='outlined' onClick={handleResetAll}>
      모든 알림 다시 표시
    </Button>

    { /* 성공 알림 */ }
    { alerts.success && (
      <Alert severity='success' title='성공' closable onClose={() => handleClose('success')}>
        작업이 성공적으로 완료되었습니다.
      </Alert>
    ) }

    { /* 오류 알림 */ }
    { alerts.error && (
      <Alert
        severity='error'
        title='오류 발생'
        variant='filled'
        closable
        onClose={() => handleClose('error')}
      >
        예상치 못한 오류가 발생했습니다. 다시 시도해주세요.
      </Alert>
    ) }

    { /* 경고 알림 */ }
    { alerts.warning && (
      <Alert
        severity='warning'
        title='주의'
        variant='outlined'
        closable
        onClose={() => handleClose('warning')}
      >

```



```

    >
      이 작업은 되돌릴 수 없습니다. 신중하게 진행해주세요.
    </Alert>
  )}

  { /* 정보 알림 */ }
  { alerts.info && (
    <Alert severity='info' closable onClose={() => handleClose('info')}>
      시스템 업데이트가 예정되어 있습니다. (2024-01-01 02:00 AM)
    </Alert>
  )}

  { /* 커스텀 액션이 있는 알림 */ }
  <Alert
    severity='info'
    title='업데이트 안내'
    action={
      <Box sx={{ display: 'flex', gap: 1 }}>
        <Button size='small' variant='outlined'>
          자세히보기
        </Button>
        <Button size='small' variant='contained'>
          지금 업데이트
        </Button>
      </Box>
    }
  >
    새로운 기능이 추가되었습니다. 지금 업데이트하세요.
  </Alert>
</Box>
);
};

```

## Loading 컴포넌트

```

const LoadingExample: React.FC = () => {
  const [showOverlay, setShowOverlay] = useState(false);
  const [showBackdrop, setShowBackdrop] = useState(false);

  const handleShowOverlay = () => {
    setShowOverlay(true);
    setTimeout(() => setShowOverlay(false), 3000);
  };

  const handleShowBackdrop = () => {
    setShowBackdrop(true);
    setTimeout(() => setShowBackdrop(false), 3000);
  };

```

```

return (
  <Box sx={{ display: 'flex', flexDirection: 'column', gap: 3 }}>
    { /* 기본 로딩 */ }
    <Box>
      <Typography variant='h6' gutterBottom>
        기본 로딩
      </Typography>
      <Box sx={{ display: 'flex', gap: 2, alignItems: 'center' }}>
        <Loading />
        <Loading size='small' />
        <Loading size='large' />
      </Box>
    </Box>

    { /* 메시지가 있는 로딩 */ }
    <Box>
      <Typography variant='h6' gutterBottom>
        메시지가 있는 로딩
      </Typography>
      <Loading message='데이터를 불러오는 중...' />
    </Box>

    { /* 선형 로딩 */ }
    <Box>
      <Typography variant='h6' gutterBottom>
        선형 로딩
      </Typography>
      <Box sx={{ display: 'flex', flexDirection: 'column', gap: 2 }}>
        <Loading variant='linear' />
        <Loading variant='linear' progress={75} message='75% 완료' />
      </Box>
    </Box>

    { /* 오버레이 및 백드롭 */ }
    <Box>
      <Typography variant='h6' gutterBottom>
        오버레이 및 백드롭
      </Typography>
      <Box sx={{ display: 'flex', gap: 2 }}>
        <Button variant='outlined' onClick={handleShowOverlay}>
          오버레이 로딩 (3초)
        </Button>
        <Button variant='outlined' onClick={handleShowBackdrop}>
          백드롭 로딩 (3초)
        </Button>
      </Box>
    </Box>

    { /* 상대 위치 컨테이너 */ }

```

```

<Box sx={{ position: 'relative', height: 200, border: '1px solid #ddd', borderRadius:
  <Typography variant='h6' sx={{ p: 2 }}>
    컨테이너 내용
  </Typography>
  {showOverlay && <Loading overlay backdrop message='처리 중...' />}
</Box>

  {/* 전체 화면 백드롭 */}
  {showBackdrop && <Loading backdrop message='잠시만 기다려주세요...' size='large' />}
</Box>
);
};

```

## Toast 컴포넌트

```

const ToastExample: React.FC = () => {
  const [toasts, setToasts] = useState({
    success: false,
    error: false,
    warning: false,
    info: false,
    custom: false,
  });

  const showToast = (type: string) => {
    setToasts(prev => ({ ...prev, [type]: true }));
  };

  const hideToast = (type: string) => {
    setToasts(prev => ({ ...prev, [type]: false }));
  };

  return (
    <Box sx={{ display: 'flex', flexDirection: 'column', gap: 2 }}>
      <Typography variant='h6' gutterBottom>
        Toast 알림 예시
      </Typography>

      <Box sx={{ display: 'flex', gap: 2, flexWrap: 'wrap' }}>
        <Button variant='outlined' onClick={() => showToast('success')}>
          성공 토스트
        </Button>
        <Button variant='outlined' onClick={() => showToast('error')}>
          오류 토스트
        </Button>
        <Button variant='outlined' onClick={() => showToast('warning')}>
          경고 토스트
        </Button>
      </Box>
    </Box>
  );
};

```

```

<Button variant='outlined' onClick={() => showToast('info')}>
  정보 토스트
</Button>
<Button variant='outlined' onClick={() => showToast('custom')}>
  커스텀 토스트
</Button>
</Box>

{/* 각종 토스트들 */}
<Toast
  open={toasts.success}
  message='작업이 성공적으로 완료되었습니다!'
  severity='success'
  onClose={() => hideToast('success')}
/>

<Toast
  open={toasts.error}
  message='오류가 발생했습니다. 다시 시도해주세요.'
  severity='error'
  onClose={() => hideToast('error')}
  autoHideDuration={10000}
/>

<Toast
  open={toasts.warning}
  message='주의: 이 작업은 되돌릴 수 없습니다.'
  severity='warning'
  onClose={() => hideToast('warning')}
  position={{ vertical: 'top', horizontal: 'center' }}
/>

<Toast
  open={toasts.info}
  message='새로운 업데이트가 있습니다.'
  severity='info'
  onClose={() => hideToast('info')}
  position={{ vertical: 'top', horizontal: 'right' }}
/>

<Toast
  open={toasts.custom}
  message='커스텀 액션이 있는 토스트입니다.'
  severity='info'
  onClose={() => hideToast('custom')}
  action={
    <Button size='small' onClick={() => console.log('Action clicked')}>
      자세히보기
    </Button>
  }
/>

```

```

    />
  </Box>
);
};

```

## 7. Layout 컴포넌트들

### Card 컴포넌트

```

const CardExample: React.FC = () => {
  const [loading, setLoading] = useState(false);

  const handleCardClick = () => {
    console.log('Card clicked');
  };

  const handleLoadingToggle = () => {
    setLoading(!loading);
  };

  return (
    <Box sx={{ display: 'flex', flexDirection: 'column', gap: 3 }}>
      <Typography variant='h6' gutterBottom>
        Card 컴포넌트 예시
      </Typography>

      <Box
        sx={{
          display: 'grid',
          gridTemplateColumns: 'repeat(auto-fit, minmax(300px, 1fr))',
          gap: 2,
        }}
      >
        {/* 기본 카드 */}
        <Card title='기본 카드' subtitle='부제목이 있는 카드'>
          <Typography variant='body2' color='text.secondary'>
            이것은 기본 카드의 예시입니다. 제목, 부제목, 그리고 본문 내용을 포함할 수 있습니다.
          </Typography>
        </Card>

        {/* 액션이 있는 카드 */}
        <Card
          title='액션 카드'
          subtitle='버튼이 있는 카드'
          actions={
            <Box sx={{ display: 'flex', gap: 1 }}>
              <Button size='small'>취소</Button>

```

```

        <Button size='small' variant='contained'>
            확인
        </Button>
    </Box>
}
>
    <Typography variant='body2' color='text.secondary'>
        이 카드는 하단에 액션 버튼들이 있습니다.
    </Typography>
</Card>

```

```

{/* 이미지가 있는 카드 */}
<Card
    title='이미지 카드'
    subtitle='미디어가 포함된 카드'
    media={{
        image: 'https://via.placeholder.com/300x200',
        alt: '예시 이미지',
        height: 200,
    }}
    actions={
        <Box sx={{ display: 'flex', gap: 1 }}>
            <Button size='small'>공유</Button>
            <Button size='small'>더보기</Button>
        </Box>
    }
>
    <Typography variant='body2' color='text.secondary'>
        이미지와 함께 표시되는 카드입니다.
    </Typography>
</Card>

```

```

{/* 클릭 가능한 카드 */}
<Card
    title='클릭 가능한 카드'
    subtitle='호버 효과가 있는 카드'
    clickable
    hoverable
    onClick={handleCardClick}
>
    <Typography variant='body2' color='text.secondary'>
        이 카드를 클릭해보세요. 호버 효과와 클릭 이벤트가 있습니다.
    </Typography>
</Card>

```

```

{/* 로딩 상태 카드 */}
<Card
    title='로딩 카드'
    subtitle='로딩 상태를 보여주는 카드'
    loading={loading}

```

```

        actions={
          <Button size='small' onClick={handleLoadingToggle}>
            {loading ? '로딩 중지' : '로딩 시작'}
          </Button>
        }
      >
        <Typography variant='body2' color='text.secondary'>
          로딩 상태를 토글할 수 있습니다.
        </Typography>
      </Card>

    { /* 커스텀 헤더 카드 */ }
    <Card
      header={
        <Box sx={{ display: 'flex', alignItems: 'center', gap: 2, p: 2 }}>
          <Avatar sx={{ bgcolor: 'primary.main' }}>U</Avatar>
          <Box>
            <Typography variant='h6'>사용자 카드</Typography>
            <Typography variant='body2' color='text.secondary'>
              커스텀 헤더
            </Typography>
          </Box>
        </Box>
      }
    >
      <Typography variant='body2' color='text.secondary'>
        커스텀 헤더를 사용한 카드입니다.
      </Typography>
    </Card>
  </Box>
</Box>
);
};

```

## Tabs 컴포넌트

```

const TabsExample: React.FC = () => {
  const [currentTab, setCurrentTab] = useState('tab1');

  const basicTabs = [
    {
      id: 'tab1',
      label: '기본 정보',
      content: (
        <Box sx={{ p: 2 }}>
          <Typography variant='h6' gutterBottom>
            기본 정보
          </Typography>

```

```

        <Typography>사용자의 기본 정보를 표시합니다.</Typography>
      </Box>
    ),
  },
  {
    id: 'tab2',
    label: '상세 정보',
    content: (
      <Box sx={{ p: 2 }}>
        <Typography variant='h6' gutterBottom>
          상세 정보
        </Typography>
        <Typography>사용자의 상세한 정보를 표시합니다.</Typography>
      </Box>
    ),
  },
  {
    id: 'tab3',
    label: '설정',
    content: (
      <Box sx={{ p: 2 }}>
        <Typography variant='h6' gutterBottom>
          설정
        </Typography>
        <Typography>사용자 설정을 관리합니다.</Typography>
      </Box>
    ),
  },
];

```

```

const iconTabs = [
  {
    id: 'dashboard',
    label: '대시보드',
    icon: <DashboardIcon />,
    content: (
      <Box sx={{ p: 2 }}>
        <Typography variant='h6' gutterBottom>
          대시보드
        </Typography>
        <Typography>시스템 개요를 확인할 수 있습니다.</Typography>
      </Box>
    ),
  },
  {
    id: 'users',
    label: '사용자',
    icon: <PeopleIcon />,
    badge: 5,
    content: (

```



```

        <Box sx={{ p: 2 }}>
          <Typography variant='h6' gutterBottom>
            사용자 관리
          </Typography>
          <Typography>사용자 목록과 관리 기능을 제공합니다.</Typography>
        </Box>
      ),
    },
    {
      id: 'settings',
      label: '설정',
      icon: <SettingsIcon />,
      content: (
        <Box sx={{ p: 2 }}>
          <Typography variant='h6' gutterBottom>
            시스템 설정
          </Typography>
          <Typography>시스템 설정을 변경할 수 있습니다.</Typography>
        </Box>
      ),
    },
    {
      id: 'help',
      label: '도움말',
      icon: <HelpIcon />,
      disabled: true,
      content: (
        <Box sx={{ p: 2 }}>
          <Typography variant='h6' gutterBottom>
            도움말
          </Typography>
          <Typography>도움말과 FAQ를 확인할 수 있습니다.</Typography>
        </Box>
      ),
    },
  ],
];

return (
  <Box sx={{ display: 'flex', flexDirection: 'column', gap: 4 }}>
    <Typography variant='h6' gutterBottom>
      Tabs 컴포넌트 예시
    </Typography>

    { /* 기본 탭 */ }
    <Paper sx={{ p: 2 }}>
      <Typography variant='subtitle1' gutterBottom>
        기본 탭
      </Typography>
      <Tabs items={basicTabs} defaultValue='tab1' />
    </Paper>
  </Box>
);

```

```

    { /* 아이콘과 배지가 있는 탭 */}
    <Paper sx={{ p: 2 }}>
      <Typography variant='subtitle1' gutterBottom>
        아이콘과 배지가 있는 탭
      </Typography>
      <Tabs
        items={iconTabs}
        defaultValue='dashboard'
        variant='scrollable'
        indicatorColor='secondary'
      />
    </Paper>

    { /* 세로 탭 */}
    <Paper sx={{ p: 2 }}>
      <Typography variant='subtitle1' gutterBottom>
        세로 탭
      </Typography>
      <Tabs
        items={basicTabs}
        defaultValue='tab1'
        orientation='vertical'
        sx={{ minHeight: 300 }}
      />
    </Paper>

    { /* 제어된 탭 */}
    <Paper sx={{ p: 2 }}>
      <Typography variant='subtitle1' gutterBottom>
        제어된 탭
      </Typography>
      <Box sx={{ mb: 2 }}>
        <Typography variant='body2' color='text.secondary'>
          현재 선택된 탭: {currentTab}
        </Typography>
      </Box>
      <Tabs items={basicTabs} value={currentTab} onChange={setCurrentTab} variant='fullWidth' />
    </Paper>
  </Box>
);
};

```

## Drawer 컴포넌트

```

const DrawerExample: React.FC = () => {
  const [drawers, setDrawers] = useState({
    left: false,

```

```

    right: false,
    top: false,
    bottom: false,
    persistent: false,
  });

const toggleDrawer = (position: string) => {
  setDrawers(prev => ({ ...prev, [position]: !prev[position] }));
};

const drawerContent = (position: string) => (
  <Box sx={{ p: 2 }}>
    <Typography variant='h6' gutterBottom>
      {position.charAt(0).toUpperCase() + position.slice(1)} Drawer
    </Typography>
    <Typography variant='body2' color='text.secondary' gutterBottom>
      이것은 {position} 위치의 drawer입니다.
    </Typography>
    <Box sx={{ mt: 2 }}>
      <Button variant='outlined' onClick={() => toggleDrawer(position)} fullWidth>
        닫기
      </Button>
    </Box>
  </Box>
);

return (
  <Box sx={{ display: 'flex', flexDirection: 'column', gap: 3 }}>
    <Typography variant='h6' gutterBottom>
      Drawer 컴포넌트 예시
    </Typography>

    <Box sx={{ display: 'flex', gap: 2, flexWrap: 'wrap' }}>
      <Button variant='outlined' onClick={() => toggleDrawer('left')}>
        Left Drawer
      </Button>
      <Button variant='outlined' onClick={() => toggleDrawer('right')}>
        Right Drawer
      </Button>
      <Button variant='outlined' onClick={() => toggleDrawer('top')}>
        Top Drawer
      </Button>
      <Button variant='outlined' onClick={() => toggleDrawer('bottom')}>
        Bottom Drawer
      </Button>
      <Button variant='outlined' onClick={() => toggleDrawer('persistent')}>
        Persistent Drawer
      </Button>
    </Box>
  </Box>
);

```

```

{ /* Left Drawer */}
<Drawer
  open={drawers.left}
  onClose={() => toggleDrawer('left')}
  anchor='left'
  title='Left Drawer '
>
  {drawerContent('left')}
</Drawer>

{ /* Right Drawer */}
<Drawer
  open={drawers.right}
  onClose={() => toggleDrawer('right')}
  anchor='right'
  title='Right Drawer '
  width={400}
  actions={
    <Button size='small' variant='contained'>
      저장
    </Button>
  }
>
  {drawerContent('right')}
  <Box sx={{ mt: 2 }}>
    <TextField label='이름' fullWidth sx={{ mb: 2 }} />
    <TextField label='이메일' fullWidth sx={{ mb: 2 }} />
    <Select
      label='역할'
      options={[
        { value: 'admin', label: '관리자' },
        { value: 'user', label: '사용자' },
      ]}
      fullWidth
    />
  </Box>
</Drawer>

{ /* Top Drawer */}
<Drawer
  open={drawers.top}
  onClose={() => toggleDrawer('top')}
  anchor='top'
  title='Top Drawer '
  width={200}
>
  {drawerContent('top')}
</Drawer>

{ /* Bottom Drawer */}

```

```

<Drawer
  open={drawers.bottom}
  onClose={() => toggleDrawer('bottom')}
  anchor='bottom'
  title='Bottom Drawer'
  width={300}
>
  {drawerContent('bottom')}
</Drawer>

{/* Persistent Drawer */}
<Drawer
  open={drawers.persistent}
  onClose={() => toggleDrawer('persistent')}
  variant='persistent'
  title='Persistent Drawer'
  hideCloseButton
  actions={
    <Button size='small' onClick={() => toggleDrawer('persistent')}>
      닫기
    </Button>
  }
>
  {drawerContent('persistent')}
</Drawer>
</Box>
);
};

```

## 8. 고급 조합 사용법

### 관리자 대시보드 예시

```

const AdminDashboard: React.FC = () => {
  const [selectedTab, setSelectedTab] = useState('users');
  const [isDrawerOpen, setIsDrawerOpen] = useState(false);
  const [isModalOpen, setIsModalOpen] = useState(false);
  const [notification, setNotification] = useState({ open: false, message: '', severity: 'info' });

  const showNotification = (
    message: string,
    severity: 'success' | 'error' | 'warning' | 'info'
  ) => {
    setNotification({ open: true, message, severity });
  };

  const dashboardTabs = [

```

```

{
  id: 'users',
  label: '사용자 관리',
  icon: <PeopleIcon />,
  badge: 12,
  content: (
    <Box sx={{ p: 3 }}>
      <Box
        sx={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center', ml
      >
        <Typography variant='h5'>사용자 관리</Typography>
        <Button
          variant='contained'
          startIcon={<AddIcon />}
          onClick={() => setIsModalOpen(true)}
        >
          사용자 추가
        </Button>
      </Box>
      <DataGrid data={users} columns={userColumns} height={400} toolbar searchable expo
    </Box>
  ),
},
{
  id: 'analytics',
  label: '분석',
  icon: <AnalyticsIcon />,
  content: (
    <Box sx={{ p: 3 }}>
      <Typography variant='h5' gutterBottom>
        분석 대시보드
      </Typography>
      <Box
        sx={{
          display: 'grid',
          gridTemplateColumns: 'repeat(auto-fit, minmax(300px, 1fr))',
          gap: 2,
        }}
      >
        <Card title='총 사용자' subtitle='이번 달'>
          <Typography variant='h3' color='primary'>
            1,234
          </Typography>
        </Card>
        <Card title='활성 사용자' subtitle='지난 7일'>
          <Typography variant='h3' color='success.main'>
            987
          </Typography>
        </Card>
        <Card title='매출' subtitle='이번 달'>

```

```

        <Typography variant='h3' color='warning.main'>
          $12,345
        </Typography>
      </Card>
    </Box>
  </Box>
),
},
{
  id: 'settings',
  label: '설정',
  icon: <SettingsIcon />,
  content: (
    <Box sx={{ p: 3 }}>
      <Typography variant='h5' gutterBottom>
        시스템 설정
      </Typography>
      <Alert severity='info' sx={{ mb: 2 }}>
        설정을 변경하면 시스템 재시작이 필요할 수 있습니다.
      </Alert>
      <Box sx={{ display: 'flex', flexDirection: 'column', gap: 2 }}>
        <TextField label='시스템 이름' defaultValue='ITCEN Solution' fullWidth />
        <TextField label='관리자 이메일' defaultValue='admin@itcen.com' fullWidth />
        <Select
          label='시간대'
          options={[
            { value: 'Asia/Seoul', label: '한국 표준시' },
            { value: 'UTC', label: '협정 세계시' },
          ]}
          defaultValue='Asia/Seoul'
          fullWidth
        />
      </Box>
    </Box>
  ),
},
];

return (
  <Box sx={{ display: 'flex', flexDirection: 'column', minHeight: '100vh' }}>
    { /* 헤더 */ }
    <Box sx={{ bgcolor: 'primary.main', color: 'white', p: 2 }}>
      <Box sx={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center' }}>
        <Typography variant='h6'>ITCEN Solution 관리자</Typography>
        <Box sx={{ display: 'flex', gap: 2 }}>
          <Button
            color='inherit'
            startIcon={<NotificationsIcon />}
            onClick={() => setIsDrawerOpen(true)}
          />
        </Box>
      </Box>
    </Box>
  );

```

```

        알림
      </Button>
      <Button color='inherit'>로그아웃</Button>
    </Box>
  </Box>
</Box>

{/* 메인 콘텐츠 */}
<Box sx={{ flex: 1 }}>
  <Tabs
    items={dashboardTabs}
    value={selectedTab}
    onChange={setSelectedTab}
    variant='scrollable'
    indicatorColor='primary'
  />
</Box>

{/* 알림 드로어 */}
<Drawer
  open={isDrawerOpen}
  onClose={() => setIsDrawerOpen(false)}
  anchor='right'
  title='알림'
  width={400}
>
  <Box sx={{ display: 'flex', flexDirection: 'column', gap: 2 }}>
    <Alert severity='info' title='새로운 사용자'>
      3명의 새로운 사용자가 등록되었습니다.
    </Alert>
    <Alert severity='warning' title='시스템 점검'>
      내일 오전 2시에 시스템 점검이 예정되어 있습니다.
    </Alert>
    <Alert severity='success' title='백업 완료'>
      데이터 백업이 성공적으로 완료되었습니다.
    </Alert>
  </Box>
</Drawer>

{/* 사용자 추가 모달 */}
<Modal
  open={isModalOpen}
  onClose={() => setIsModalOpen(false)}
  title='새 사용자 추가'
  maxWidth='md'
  actions={
    <Box sx={{ display: 'flex', gap: 1 }}>
      <Button onClick={() => setIsModalOpen(false)}>취소</Button>
      <Button
        variant='contained'

```



```

        onClick={() => {
          setIsModalOpen(false);
          showNotification('사용자가 성공적으로 추가되었습니다.', 'success');
        }}
      >
        추가
      </Button>
    </Box>
  }
}
>
<Box sx={{ display: 'flex', flexDirection: 'column', gap: 2, pt: 2 }}>
  <TextField label='이름' required fullWidth />
  <TextField label='이메일' type='email' required fullWidth />
  <Select
    label='역할'
    options={[
      { value: 'admin', label: '관리자' },
      { value: 'user', label: '일반 사용자' },
      { value: 'guest', label: '게스트' },
    ]}
    required
    fullWidth
  />
  <TextField label='전화번호' fullWidth />
</Box>
</Modal>

{/* 토스트 알림 */}
<Toast
  open={notification.open}
  message={notification.message}
  severity={notification.severity}
  onClose={() => setNotification({ ...notification, open: false })}
/>
</Box>
);
};

```

이렇게 다양한 컴포넌트들을 조합하여 복잡하고 기능적인 사용자 인터페이스를 구성할 수 있습니다.

## 9. Navigation 컴포넌트들

### Breadcrumb 컴포넌트

```

const BreadcrumbExample: React.FC = () => {
  const basicBreadcrumbs = [

```

```

    { id: 'home', label: '홈', href: '/', icon: <HomeIcon /> },
    { id: 'users', label: '사용자 관리', href: '/users' },
    { id: 'detail', label: '사용자 상세' },
  ];

const longBreadcrumbs = [
  { id: 'home', label: '홈', href: '/' },
  { id: 'dashboard', label: '대시보드', href: '/dashboard' },
  { id: 'analytics', label: '분석', href: '/analytics' },
  { id: 'reports', label: '리포트', href: '/reports' },
  { id: 'monthly', label: '월간 리포트', href: '/reports/monthly' },
  { id: 'detail', label: '2024년 1월' },
];

const handleBreadcrumbClick = (item: any) => {
  console.log('Breadcrumb clicked:', item);
};

return (
  <Box sx={{ display: 'flex', flexDirection: 'column', gap: 3 }}>
    <Typography variant='h6' gutterBottom>
      Breadcrumb 컴포넌트 예시
    </Typography>

    { /* 기본 브레드크럼 */ }
    <Paper sx={{ p: 2 }}>
      <Typography variant='subtitle1' gutterBottom>
        기본 브레드크럼
      </Typography>
      <Breadcrumb items={basicBreadcrumbs} />
    </Paper>

    { /* 축약된 브레드크럼 */ }
    <Paper sx={{ p: 2 }}>
      <Typography variant='subtitle1' gutterBottom>
        축약된 브레드크럼
      </Typography>
      <Breadcrumb
        items={longBreadcrumbs}
        maxItems={4}
        itemsBeforeCollapse={2}
        itemsAfterCollapse={2}
      />
    </Paper>

    { /* 클릭 이벤트가 있는 브레드크럼 */ }
    <Paper sx={{ p: 2 }}>
      <Typography variant='subtitle1' gutterBottom>
        클릭 이벤트
      </Typography>

```

```

        <Breadcrumbs
          items={basicBreadcrumbs.map(item => ({
            ...item,
            onClick: () => handleBreadcrumbClick(item),
          }))}
        />
      </Paper>

      { /* 커스텀 구분자 */ }
      <Paper sx={{ p: 2 }}>
        <Typography variant='subtitle1' gutterBottom>
          커스텀 구분자
        </Typography>
        <Breadcrumbs items={basicBreadcrumbs} separator='>' />
      </Paper>
    </Box>
  );
};

```

## Pagination 컴포넌트

```

const PaginationExample: React.FC = () => {
  const [page1, setPage1] = useState(1);
  const [page2, setPage2] = useState(5);
  const [page3, setPage3] = useState(10);

  return (
    <Box sx={{ display: 'flex', flexDirection: 'column', gap: 3 }}>
      <Typography variant='h6' gutterBottom>
        Pagination 컴포넌트 예시
      </Typography>

      { /* 기본 페이지네이션 */ }
      <Paper sx={{ p: 2 }}>
        <Typography variant='subtitle1' gutterBottom>
          기본 페이지네이션
        </Typography>
        <Pagination count={10} page={page1} onChange={setPage1} />
        <Typography variant='body2' color='text.secondary' sx={{ mt: 1 }}>
          현재 페이지: {page1}
        </Typography>
      </Paper>

      { /* 아웃라인 스타일 */ }
      <Paper sx={{ p: 2 }}>
        <Typography variant='subtitle1' gutterBottom>
          아웃라인 스타일
        </Typography>

```

```

    <Pagination
      count={20}
      page={page2}
      onChange={setPage2}
      variant='outlined'
      shape='rounded'
      size='large'
      showFirstButton
      showLastButton
    />
    <Typography variant='body2' color='text.secondary' sx={{ mt: 1 }}>
      현재 페이지: {page2}
    </Typography>
  </Paper>

  { /* 컴팩트 버전 */ }
  <Paper sx={{ p: 2 }}>
    <Typography variant='subtitle1' gutterBottom>
      컴팩트 버전
    </Typography>
    <Pagination
      count={50}
      page={page3}
      onChange={setPage3}
      size='small'
      siblingCount={1}
      boundaryCount={1}
      color='secondary'
    />
    <Typography variant='body2' color='text.secondary' sx={{ mt: 1 }}>
      현재 페이지: {page3}
    </Typography>
  </Paper>
</Box>
);
};

```

## Stepper 컴포넌트

```

const StepperExample: React.FC = () => {
  const [activeStep, setActiveStep] = useState(1);
  const [verticalActiveStep, setVerticalActiveStep] = useState(0);

  const steps = [
    {
      id: 'step1',
      label: '기본 정보 입력',
      description: '사용자의 기본 정보를 입력합니다.',
    },
  ],

```

```

        completed: true,
      },
      {
        id: 'step2',
        label: '상세 정보 입력',
        description: '추가 정보를 입력합니다.',
      },
      {
        id: 'step3',
        label: '확인',
        description: '입력된 정보를 확인합니다.',
        optional: true,
        optionalLabel: '선택사항',
      },
    ],
  ];

```

```

const iconSteps = [
  {
    id: 'account',
    label: '계정 생성',
    icon: <PersonIcon />,
    completed: true,
  },
  {
    id: 'profile',
    label: '프로필 설정',
    icon: <SettingsIcon />,
  },
  {
    id: 'verification',
    label: '이메일 인증',
    icon: <EmailIcon />,
  },
  {
    id: 'complete',
    label: '완료',
    icon: <CheckIcon />,
  },
];

```

```

const handleStepClick = (stepIndex: number) => {
  setActiveStep(stepIndex);
};

```

```

const handleVerticalStepClick = (stepIndex: number) => {
  setVerticalActiveStep(stepIndex);
};

```

```

return (
  <Box sx={{ display: 'flex', flexDirection: 'column', gap: 4 }}>

```

```
<Typography variant='h6' gutterBottom>
```

```
  Stepper 컴포넌트 예시
```

```
</Typography>
```

```
{/* 기본 수평 스텝퍼 */}
```

```
<Paper sx={{ p: 3 }}>
```

```
  <Typography variant='subtitle1' gutterBottom>
```

```
    기본 수평 스텝퍼
```

```
</Typography>
```

```
<Stepper steps={steps} activeStep={activeStep} onStepClick={handleStepClick} nonLinear />
```

```
<Box sx={{ mt: 2, display: 'flex', gap: 1 }}>
```

```
  <Button
```

```
    variant='outlined'
```

```
    onClick={() => setActiveStep(Math.max(0, activeStep - 1))}
```

```
    disabled={activeStep === 0}
```

```
>
```

```
    이전
```

```
</Button>
```

```
<Button
```

```
  variant='contained'
```

```
  onClick={() => setActiveStep(Math.min(steps.length - 1, activeStep + 1))}
```

```
  disabled={activeStep === steps.length - 1}
```

```
>
```

```
  다음
```

```
</Button>
```

```
</Box>
```

```
</Paper>
```

```
{/* 아이콘 스텝퍼 */}
```

```
<Paper sx={{ p: 3 }}>
```

```
  <Typography variant='subtitle1' gutterBottom>
```

```
    아이콘 스텝퍼
```

```
</Typography>
```

```
<Stepper steps={iconSteps} activeStep={1} alternativeLabel />
```

```
</Paper>
```

```
{/* 세로 스텝퍼 */}
```

```
<Paper sx={{ p: 3 }}>
```

```
  <Typography variant='subtitle1' gutterBottom>
```

```
    세로 스텝퍼
```

```
</Typography>
```

```
<Stepper
```

```
  steps={steps}
```

```
  activeStep={verticalActiveStep}
```

```
  orientation='vertical'
```

```
  onStepClick={handleVerticalStepClick}
```

```
  nonLinear
```

```
/>
```

```
</Paper>
```

```
</Box>
```

```
);  
};
```

## 10. Data Display 확장 컴포넌트들

### Badge 컴포넌트

```
const BadgeExample: React.FC = () => {  
  return (  
    <Box sx={{ display: 'flex', flexDirection: 'column', gap: 3 }}>  
      <Typography variant='h6' gutterBottom>  
        Badge 컴포넌트 예시  
      </Typography>  
  
      <Box sx={{ display: 'flex', gap: 4, flexWrap: 'wrap', alignItems: 'center' }}>  
        {/* 기본 배지 */}  
        <Badge badgeContent={4} color='primary'>  
          <MailIcon />  
        </Badge>  
  
        {/* 닷 배지 */}  
        <Badge variant='dot' color='error'>  
          <NotificationsIcon />  
        </Badge>  
  
        {/* 최대값 제한 */}  
        <Badge badgeContent={1000} max={99} color='secondary'>  
          <MailIcon />  
        </Badge>  
  
        {/* 커스텀 위치 */}  
        <Badge  
          badgeContent='NEW'  
          color='success'  
          anchorOrigin={{ vertical: 'bottom', horizontal: 'right' }}  
        >  
          <Button variant='outlined'>업데이트</Button>  
        </Badge>  
  
        {/* 다양한 색상 */}  
        <Badge badgeContent={5} color='warning'>  
          <ShoppingCartIcon />  
        </Badge>  
  
        <Badge badgeContent='HOT' color='error'>  
          <LocalFireDepartmentIcon />  
        </Badge>
```

```

    </Box>

    { /* 숨김 처리 */ }
    <Box sx={{ display: 'flex', gap: 2, alignItems: 'center' }}>
      <Badge badgeContent={0} showZero color='primary'>
        <MailIcon />
      </Badge>
      <Badge badgeContent={0} color='primary'>
        <MailIcon />
      </Badge>
      <Typography variant='body2' color='text.secondary'>
        showZero 속성 비교
      </Typography>
    </Box>
  </Box>
);
};

```

## Chip 컴포넌트

```

const ChipExample: React.FC = () => {
  const [chips, setChips] = useState([
    { id: 1, label: 'React', deletable: true },
    { id: 2, label: 'TypeScript', deletable: true },
    { id: 3, label: 'Material-UI', deletable: true },
  ]);

  const handleDelete = (chipId: number) => {
    setChips(chips.filter(chip => chip.id !== chipId));
  };

  const handleClick = (label: string) => {
    console.log(`${label} 클릭됨`);
  };

  return (
    <Box sx={{ display: 'flex', flexDirection: 'column', gap: 3 }}>
      <Typography variant='h6' gutterBottom>
        Chip 컴포넌트 예시
      </Typography>

      { /* 기본 칩들 */ }
      <Paper sx={{ p: 2 }}>
        <Typography variant='subtitle1' gutterBottom>
          기본 칩
        </Typography>
        <Box sx={{ display: 'flex', gap: 1, flexWrap: 'wrap' }}>
          <Chip label='기본' />

```



```

    <Chip label='Primary' color='primary' />
    <Chip label='Secondary' color='secondary' />
    <Chip label='Success' color='success' />
    <Chip label='Error' color='error' />
    <Chip label='Warning' color='warning' />
    <Chip label='Info' color='info' />
  </Box>
</Paper>

```

{/\* 아웃라인 칩들 \*/}

```

<Paper sx={{ p: 2 }}>
  <Typography variant='subtitle1' gutterBottom>
    아웃라인 칩
  </Typography>
  <Box sx={{ display: 'flex', gap: 1, flexWrap: 'wrap' }}>
    <Chip label='아웃라인' variant='outlined' />
    <Chip label='Primary' variant='outlined' color='primary' />
    <Chip label='Secondary' variant='outlined' color='secondary' />
    <Chip label='Success' variant='outlined' color='success' />
  </Box>
</Paper>

```

{/\* 아이콘과 아바타 \*/}

```

<Paper sx={{ p: 2 }}>
  <Typography variant='subtitle1' gutterBottom>
    아이콘과 아바타
  </Typography>
  <Box sx={{ display: 'flex', gap: 1, flexWrap: 'wrap' }}>
    <Chip icon={<Facelcon />} label='아이콘 칩' color='primary' />
    <Chip avatar={<Avatar>M</Avatar>} label='아바타 칩' variant='outlined' />
    <Chip avatar={<Avatar src='/avatar.jpg' />} label='이미지 아바타' color='secondary' />
  </Box>
</Paper>

```

{/\* 삭제 가능한 칩들 \*/}

```

<Paper sx={{ p: 2 }}>
  <Typography variant='subtitle1' gutterBottom>
    삭제 가능한 칩
  </Typography>
  <Box sx={{ display: 'flex', gap: 1, flexWrap: 'wrap' }}>
    {chips.map(chip => (
      <Chip
        key={chip.id}
        label={chip.label}
        onDelete={() => handleDelete(chip.id)}
        color='primary'
        variant='outlined'
      />
    ))}
  </Box>

```

```

</Paper>

{/* 클릭 가능한 칩들 */}
<Paper sx={{ p: 2 }}>
  <Typography variant='subtitle1' gutterBottom>
    클릭 가능한 칩
  </Typography>
  <Box sx={{ display: 'flex', gap: 1, flexWrap: 'wrap' }}>
    <Chip
      label='클릭 가능'
      onClick={() => handleClick('클릭 가능')}
      clickable
      color='success'
    />
    <Chip label='링크 칩' component='a' href='#' clickable color='info' />
  </Box>
</Paper>

{/* 크기 */}
<Paper sx={{ p: 2 }}>
  <Typography variant='subtitle1' gutterBottom>
    크기
  </Typography>
  <Box sx={{ display: 'flex', gap: 1, alignItems: 'center', flexWrap: 'wrap' }}>
    <Chip label='Small' size='small' color='primary' />
    <Chip label='Medium' size='medium' color='primary' />
  </Box>
</Paper>
</Box>
);
};

```

## 11. Form 확장 컴포넌트들

### DatePicker 컴포넌트

```

const DatePickerExample: React.FC = () => {
  const [selectedDate, setSelectedDate] = useState<Date | null>(null);
  const [birthDate, setBirthDate] = useState<Date | null>(null);
  const [reservationDate, setReservationDate] = useState<Date | null>(null);

  return (
    <Box sx={{ display: 'flex', flexDirection: 'column', gap: 3 }}>
      <Typography variant='h6' gutterBottom>
        DatePicker 컴포넌트 예시
      </Typography>

```

```

{ /* 기본 날짜 선택기 */ }
<Paper sx={{ p: 2 }}>
  <Typography variant='subtitle1' gutterBottom>
    기본 날짜 선택기
  </Typography>
  <DatePicker value={selectedDate} onChange={setSelectedDate} label='날짜 선택' fullWidth />
</Paper>

```

```

{ /* 제한된 날짜 범위 */ }
<Paper sx={{ p: 2 }}>
  <Typography variant='subtitle1' gutterBottom>
    제한된 날짜 범위 (미래 날짜만)
  </Typography>
  <DatePicker
    value={reservationDate}
    onChange={setReservationDate}
    label='예약 날짜'
    minDate={new Date()}
    disablePast
    format='yyyy년 MM월 dd일'
    helperText='오늘 이후 날짜만 선택 가능합니다'
    fullWidth
  />
</Paper>

```

```

{ /* 생년월일 선택 */ }
<Paper sx={{ p: 2 }}>
  <Typography variant='subtitle1' gutterBottom>
    생년월일 선택
  </Typography>
  <DatePicker
    value={birthDate}
    onChange={setBirthDate}
    label='생년월일'
    format='yyyy-MM-dd'
    maxDate={new Date()}
    disableFuture
    views={['year', 'month', 'day']}
    openTo='year'
    fullWidth
  />
</Paper>

```

```

{ /* 선택된 날짜 표시 */ }
<Paper sx={{ p: 2 }}>
  <Typography variant='subtitle1' gutterBottom>
    선택된 날짜들
  </Typography>
  <Box sx={{ display: 'flex', flexDirection: 'column', gap: 1 }}>
    <Typography variant='body2'>

```

```

        기본 날짜: {selectedDate ? selectedDate.toLocaleDateString('ko-KR') : '선택되지 않음'}
      </Typography>
      <Typography variant='body2'>
        예약 날짜: {' '}
        {reservationDate ? reservationDate.toLocaleDateString('ko-KR') : '선택되지 않음'}
      </Typography>
      <Typography variant='body2'>
        생년월일: {birthDate ? birthDate.toLocaleDateString('ko-KR') : '선택되지 않음'}
      </Typography>
    </Box>
  </Paper>
</Box>
);
};

```

## FileUpload 컴포넌트

```

const FileUploadExample: React.FC = () => {
  const [selectedFiles, setSelectedFiles] = useState<File[]>([]);
  const [imageFiles, setImageFiles] = useState<File[]>([]);
  const [documentFiles, setDocumentFiles] = useState<File[]>([]);
  const [uploadProgress, setUploadProgress] = useState<number[]>([]);

  const handleFileSelect = (files: File[]) => {
    setSelectedFiles(files);
    // 가짜 업로드 진행률 시뮬레이션
    setUploadProgress(files.map(() => 0));
    files.forEach((_, index) => {
      const interval = setInterval(() => {
        setUploadProgress(prev => {
          const newProgress = [...prev];
          newProgress[index] = Math.min(100, newProgress[index] + 10);
          if (newProgress[index] >= 100) {
            clearInterval(interval);
          }
          return newProgress;
        });
      }, 200);
    });
  };

  const handleFileRemove = (index: number) => {
    setSelectedFiles(prev => prev.filter((_, i) => i !== index));
    setUploadProgress(prev => prev.filter((_, i) => i !== index));
  };

  const handleImageSelect = (files: File[]) => {
    setImageFiles(files);
  };

```

```

};

const handleImageRemove = (index: number) => {
  setImageFiles(prev => prev.filter((_, i) => i !== index));
};

const handleDocumentSelect = (files: File[]) => {
  setDocumentFiles(files);
};

const handleDocumentRemove = (index: number) => {
  setDocumentFiles(prev => prev.filter((_, i) => i !== index));
};

const handleError = (error: string) => {
  console.error('File upload error:', error);
};

return (
  <Box sx={{ display: 'flex', flexDirection: 'column', gap: 3 }}>
    <Typography variant='h6' gutterBottom>
      FileUpload 컴포넌트 예시
    </Typography>

    { /* 기본 파일 업로드 */ }
    <Paper sx={{ p: 2 }}>
      <Typography variant='subtitle1' gutterBottom>
        기본 파일 업로드
      </Typography>
      <FileUpload
        onFileSelect={handleFileSelect}
        files={selectedFiles}
        onFileRemove={handleFileRemove}
        uploadProgress={uploadProgress}
        maxSize={5 * 1024 * 1024} // 5MB
        multiple
        label='파일 선택'
        helperText='최대 5MB까지 업로드 가능합니다'
      />
    </Paper>

    { /* 이미지 전용 드래그 앤 드롭 */ }
    <Paper sx={{ p: 2 }}>
      <Typography variant='subtitle1' gutterBottom>
        이미지 드래그 앤 드롭
      </Typography>
      <FileUpload
        variant='dropzone'
        accept='image/*'
        multiple

```

```

        maxFiles={5}
        onFileSelect={handleImageSelect}
        files={imageFiles}
        onFileRemove={handleImageRemove}
        preview
        dropzoneText='이미지를 드래그하거나 클릭하여 업로드'
        onError={handleError}
      />
    </Paper>

    { /* 문서 파일 업로드 */ }
    <Paper sx={{ p: 2 }}>
      <Typography variant='subtitle1' gutterBottom>
        문서 파일 업로드
      </Typography>
      <FileUpload
        onFileSelect={handleDocumentSelect}
        files={documentFiles}
        onFileRemove={handleDocumentRemove}
        accept='.pdf,.doc,.docx,.xls,.xlsx,.ppt,.pptx'
        allowedFileTypes={['pdf', 'doc', 'docx', 'xls', 'xlsx', 'ppt', 'pptx']}
        maxSize={10 * 1024 * 1024} // 10MB
        multiple
        buttonText='문서 선택'
        showFileList={false} // 칩 형태로 표시
        label='문서 파일'
        helperText='PDF, Word, Excel, PowerPoint 파일만 업로드 가능합니다'
      />
    </Paper>

    { /* 단일 파일 업로드 */ }
    <Paper sx={{ p: 2 }}>
      <Typography variant='subtitle1' gutterBottom>
        단일 파일 업로드
      </Typography>
      <FileUpload
        variant='dropzone'
        onFileSelect={files => console.log('Single file:', files[0])}
        multiple={false}
        dropzoneText='하나의 파일만 업로드 가능합니다'
        maxSize={2 * 1024 * 1024} // 2MB
      />
    </Paper>
  </Box>
);
};

```

## 문제 해결 사례 및 Best Practices

# 1. DataGrid 높이 문제 해결 케이스

## 문제 상황:

MUI X: useResizeContainer - The parent DOM element of the Data Grid has an empty height. Please make sure that this element has an intrinsic height. The grid displays with a height of 0px.

## 해결 과정:

### 1. 1단계: 복잡한 Flex 레이아웃 시도

```
// ❌ 복잡하고 불안정한 해결책
const ComplexFlexSolution: React.FC = () => {
  return (
    <Box sx={{
      display: 'flex',
      flexDirection: 'column',
      height: '100vh',
      minHeight: 0
    }}>
      <Paper sx={{
        flex: 1,
        display: 'flex',
        flexDirection: 'column',
        minHeight: 0
      }}>
        <DataGrid
          data={data}
          columns={columns}
          height="100%"
          sx={{ flex: 1 }}
        />
      </Paper>
    </Box>
  );
};
```

### 2. 2단계: 고정 높이 설정

```
// ⚠️ 부분적 해결책 (반응형 문제)
const FixedHeightSolution: React.FC = () => {
  return (
    <DataGrid
      data={data}
      columns={columns}
      height={600} // 고정 높이
    />
  );
};
```

```
    />  
  );  
};
```

### 3. 3단계: 동적 높이 계산

```
// ⚠️ 복잡하고 성능 이슈  
const DynamicHeightSolution: React.FC = () => {  
  const [gridHeight, setGridHeight] = useState(400);  
  const containerRef = useRef<HTMLDivElement>(null);  
  
  useEffect(() => {  
    const updateHeight = () => {  
      if (containerRef.current) {  
        const availableHeight = window.innerHeight - 200;  
        setGridHeight(Math.max(400, availableHeight));  
      }  
    };  
  
    updateHeight();  
    window.addEventListener('resize', updateHeight);  
    return () => window.removeEventListener('resize', updateHeight);  
  }, []);  
  
  return (  
    <Box ref={containerRef}>  
      <DataGrid  
        data={data}  
        columns={columns}  
        height={gridHeight}  
      />  
    </Box>  
  );  
};
```

### 4. 4단계: 최종 해결책 - autoHeight 사용

```
// ✅ 간단하고 안정적인 해결책  
const AutoHeightSolution: React.FC = () => {  
  return (  
    <DataGrid  
      data={data}  
      columns={columns}  
      autoHeight // 이것만으로 모든 문제 해결!  
    />  
  );  
};
```



## 2. 글로벌 상태 관리 시스템 활용

### Toast와 Loading의 효율적 조합:

```
const OptimizedApiCall: React.FC = () => {
  const { showToast } = useToast();
  const { showLoading, hideLoading } = useLoading();

  const handleComplexOperation = async () => {
    showLoading('복잡한 작업을 수행하는 중...');

    try {
      // 1단계: 데이터 검증
      showLoading('데이터를 검증하는 중...');
      await validateData();

      // 2단계: 파일 업로드
      showLoading('파일을 업로드하는 중...');
      await uploadFiles();

      // 3단계: 데이터 저장
      showLoading('데이터를 저장하는 중...');
      await saveData();

      // 성공 메시지
      showToast('모든 작업이 성공적으로 완료되었습니다!', 'success');

    } catch (error) {
      console.error('작업 실패:', error);
      showToast('작업 중 오류가 발생했습니다. 다시 시도해주세요.', 'error');
    } finally {
      hideLoading();
    }
  };

  return (
    <Button onClick={handleComplexOperation} variant="contained">
      복잡한 작업 실행
    </Button>
  );
};
```

## 3. 성능 최적화 Best Practices

### 메모이제이션을 활용한 DataGrid 최적화:

```

const OptimizedDataGrid: React.FC = () => {
  const [data, setData] = useState<User[]>([]);
  const [filters, setFilters] = useState({});

  // 컬럼 정의 메모이제이션
  const columns = useMemo(() => [
    {
      field: 'id',
      headerName: 'ID',
      width: 70,
    },
    {
      field: 'name',
      headerName: '이름',
      width: 120,
      renderCell: ({ value, row }) => (
        <Chip
          label={value}
          color={row.status === 'active' ? 'success' : 'default'}
          size="small"
        />
      ),
    },
    // ... 다른 컬럼들
  ], []);

  // 필터링된 데이터 메모이제이션
  const filteredData = useMemo(() => {
    return data.filter(item => {
      // 필터 로직
      return true;
    });
  }, [data, filters]);

  // 이벤트 핸들러 메모이제이션
  const handleRowClick = useCallback((row: User) => {
    console.log('선택된 행:', row);
    navigate(`/users/${row.id}`);
  }, [navigate]);

  return (
    <DataGrid
      data={filteredData}
      columns={columns}
      onRowClick={handleRowClick}
      autoHeight
      searchable
      filterable
    />
  );
};

```

```
);  
};
```

## 4. 에러 처리 Best Practices

### 포괄적인 에러 처리:

```
const RobustComponent: React.FC = () => {  
  const { showToast } = useToast();  
  const { showLoading, hideLoading } = useLoading();  
  
  const handleApiCall = async () => {  
    showLoading('데이터를 처리하는 중...');  
  
    try {  
      const response = await api.getData();  
  
      if (!response.success) {  
        throw new Error(response.message || '서버 오류가 발생했습니다.');      }  
  
      showToast('데이터가 성공적으로 로드되었습니다.', 'success');  
  
    } catch (error) {  
      // 에러 타입별 처리  
      if (error instanceof NetworkError) {  
        showToast('네트워크 연결을 확인해주세요.', 'error');  
      } else if (error instanceof ValidationError) {  
        showToast('입력 데이터를 확인해주세요.', 'warning');  
      } else {  
        showToast('알 수 없는 오류가 발생했습니다.', 'error');  
      }  
  
      // 로깅  
      console.error('API 호출 오류:', error);  
  
    } finally {  
      hideLoading();  
    }  
  };  
  
  return (  
    <Button onClick={handleApiCall}>  
      데이터 로드  
    </Button>  
  );  
};
```

# 결론 및 핵심 가이드라인

## 주요 성과

1. **DataGrid 높이 문제 완전 해결:** `autoHeight` 속성으로 MUI X v7.7.1의 모든 높이 관련 문제 해결
2. **글로벌 상태 관리:** Toast와 Loading 시스템으로 일관된 사용자 경험 제공
3. **서버 통합 컴포넌트:** ServerDataGrid, ServerFileUpload로 실무 즉시 적용 가능
4. **성능 최적화:** 메모이제이션과 적절한 상태 관리로 성능 향상
5. **접근성 고려:** 모든 사용자가 접근 가능한 UI 구성

## 핵심 원칙

1. **단순함이 최고:** 복잡한 해결책보다는 간단하고 안정적인 방법 선택
2. **실무 중심:** 실제 프로젝트에서 발생한 문제를 바탕으로 한 실용적 솔루션
3. **일관성 유지:** 모든 컴포넌트에서 동일한 패턴과 규칙 적용
4. **확장성 고려:** 미래의 요구사항 변화에 대응할 수 있는 유연한 구조
5. **사용자 경험 우선:** 개발자 편의보다는 최종 사용자 경험을 우선 고려

## 다음 단계

- **테스트 커버리지 확대:** 각 컴포넌트에 대한 단위 테스트 및 통합 테스트 작성
- **Storybook 문서화:** 시각적 컴포넌트 가이드 및 인터랙티브 문서 작성
- **성능 모니터링:** 실제 사용 환경에서의 성능 측정 및 최적화
- **접근성 테스트:** 스크린 리더 및 키보드 네비게이션 테스트 강화

이제 ITCEN Solution 프로젝트에서 사용할 수 있는 포괄적이고 실용적인 UI 컴포넌트 라이브러리가 완성되었습니다. 각 컴포넌트는 실제 업무에서 발생한 문제들을 해결하면서 얻은 경험을 바탕으로 만들어진 검증된 솔루션입니다.